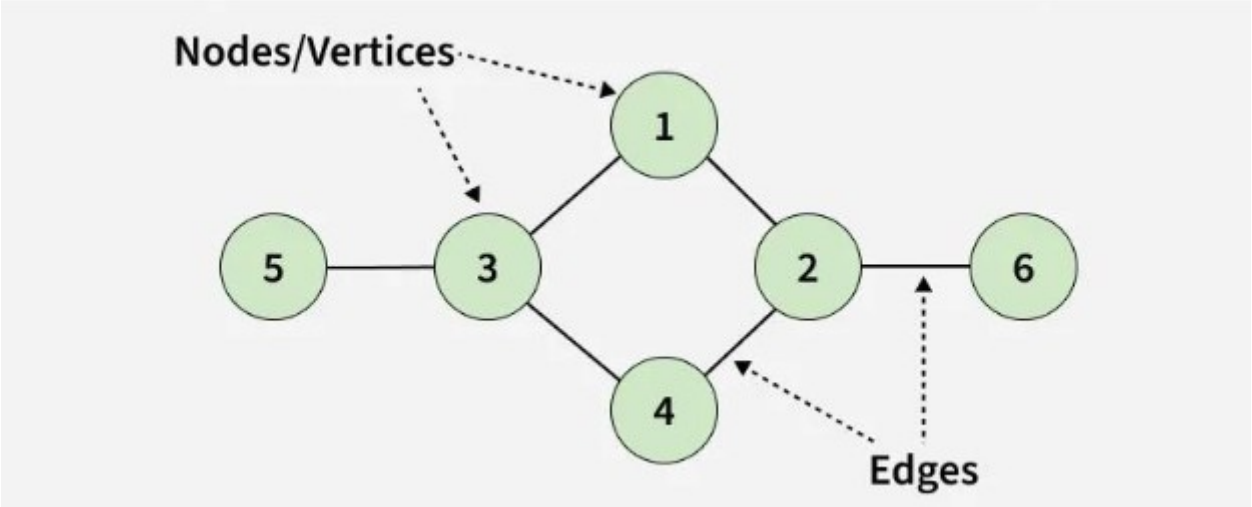
Lecture 13

Graphs II



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group
- Project 1 Final Report due **Monday** @ 11:59pm
  - Presentations on Monday (7 to 10 minutes)
  - See example project on Github
  - You received feedback via email
- Exam 1 rewrites due next Thursday at the beginning of lecture via email
  - Can earn back up to 50% on any question
  - Provide a very thorough answer and write up in a markdown file

Last time

- Graphs & Trees
- Traversals
- Breadth-First Search (BFS)

Today

- Depth-First search (DFS)
- Directed graphs
- Topological Sort

Traversal Strategies

Reachability Problem

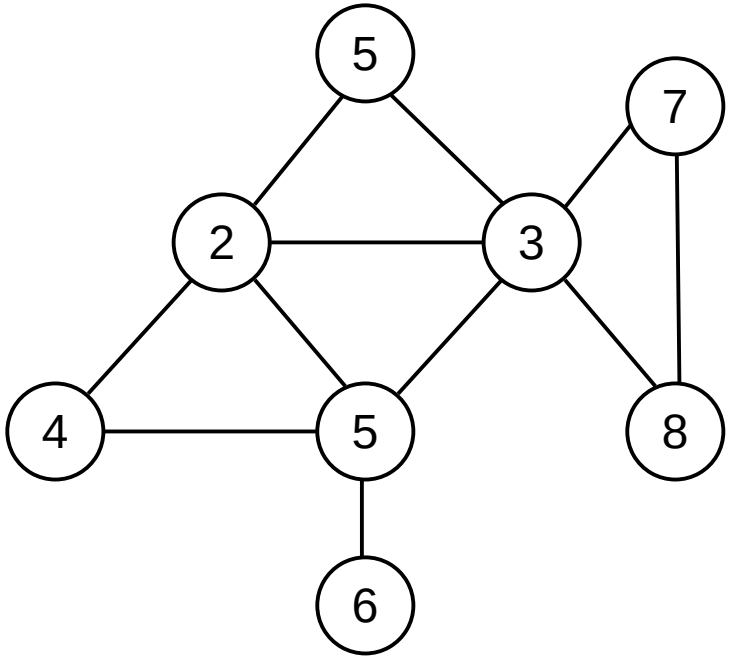
- Starting from vertex u, can we reach vertex v?

Breadth-First Search (BFS)

- Explore locally

Depth-First Search (DFS)

- Deep dive and backtrack

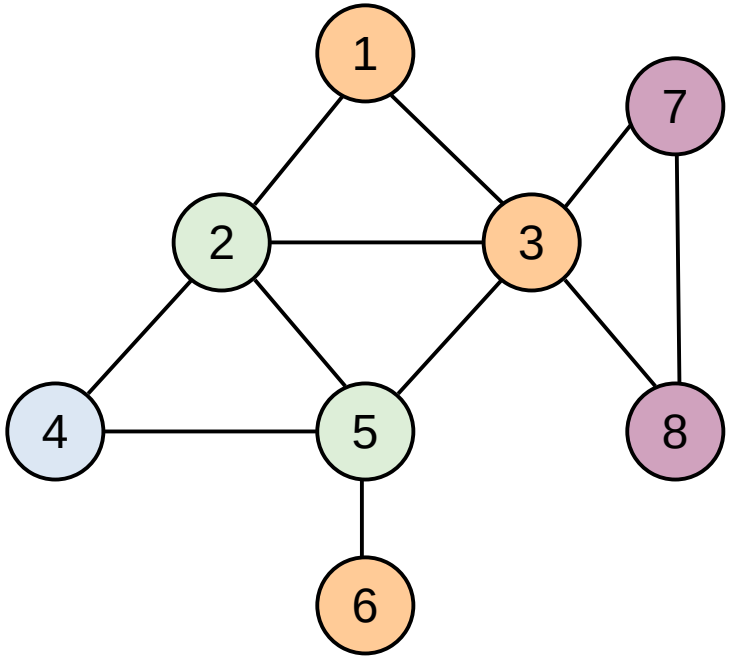


Breadth-First Search

- Let L_0 denote the set containing the root
- Define L_{i+1} from L_i as the neighbors of vertices in L_i that have not yet been visited
- Continue until no new vertices are found

Complexity: $O(n+m)$

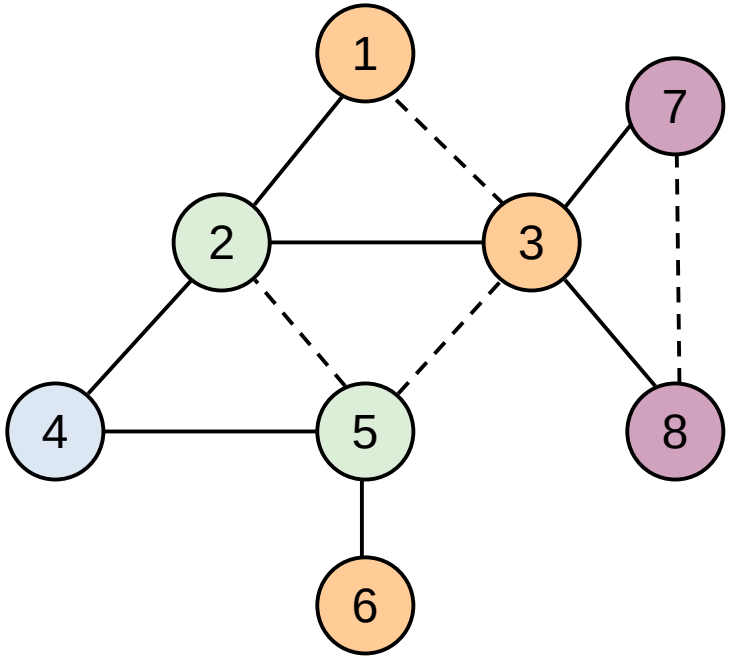
- # vertices + # edges
- Efficiently implement using a queue



BFS Tree

We can use BFS to make a tree

- Keep edge uv if v was first visited as a neighbor of u
- Each path from the starting vertex r to any other vertex v is a shortest rv path in G
- A BFS tree is not necessarily unique
  - Depend on starting vertex
  - Any choices about the order of vertices to explore

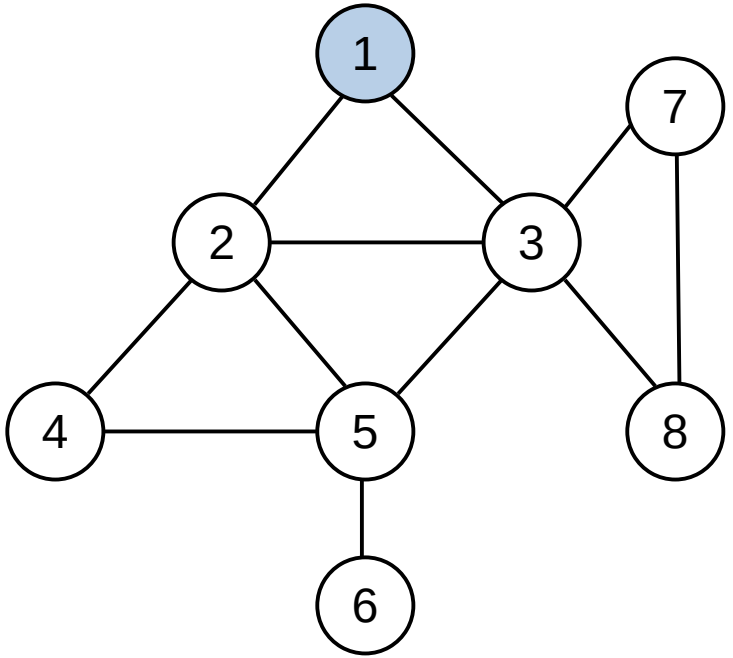


Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1

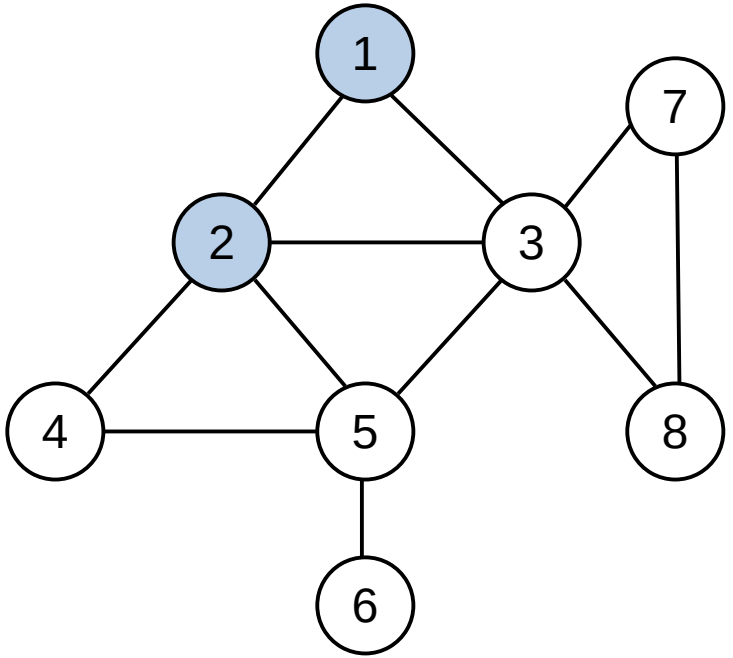


Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2

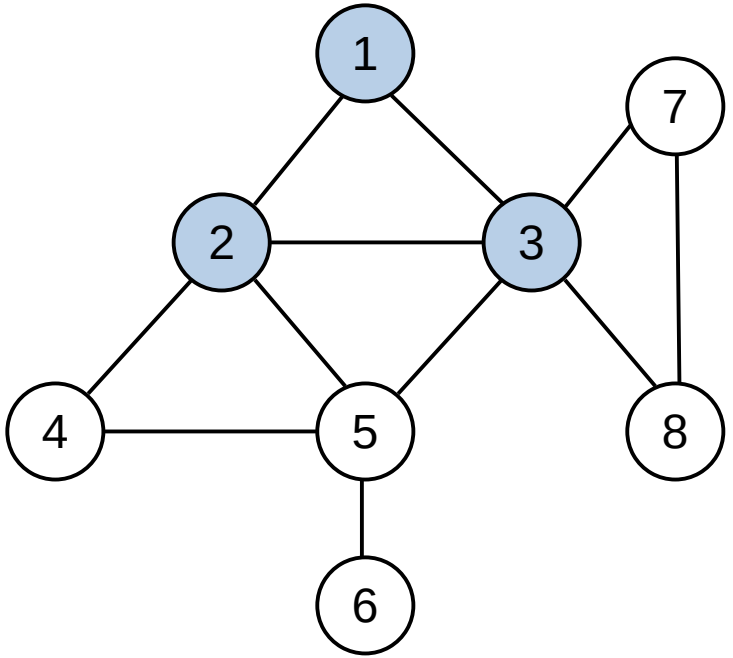


Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3

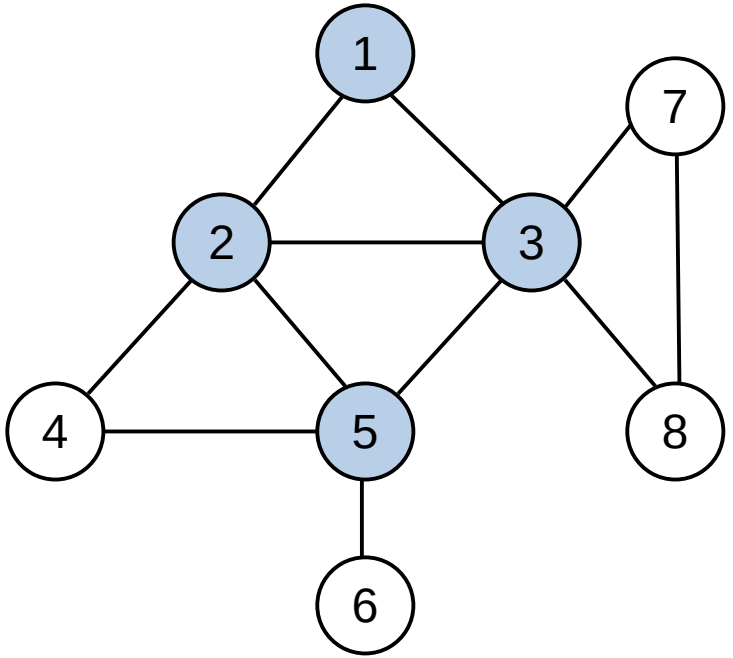


Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, 5

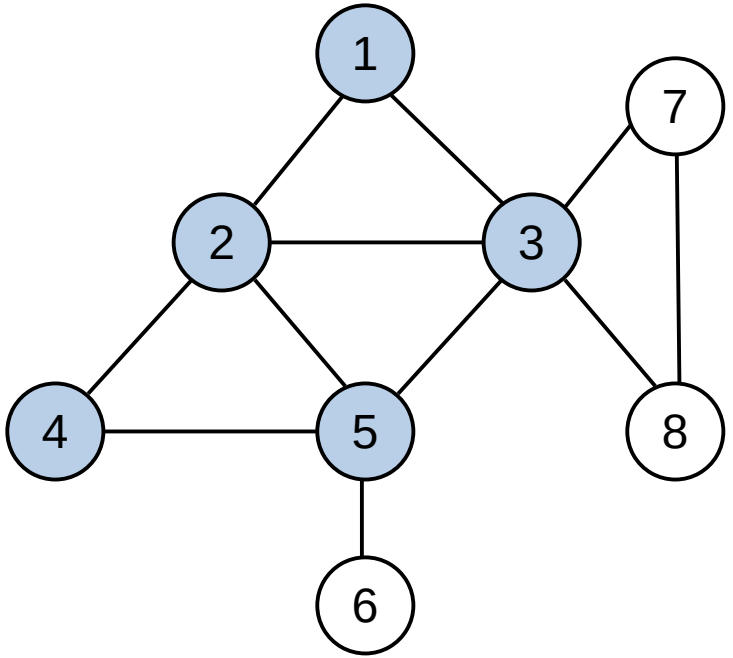


Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, 5, 4



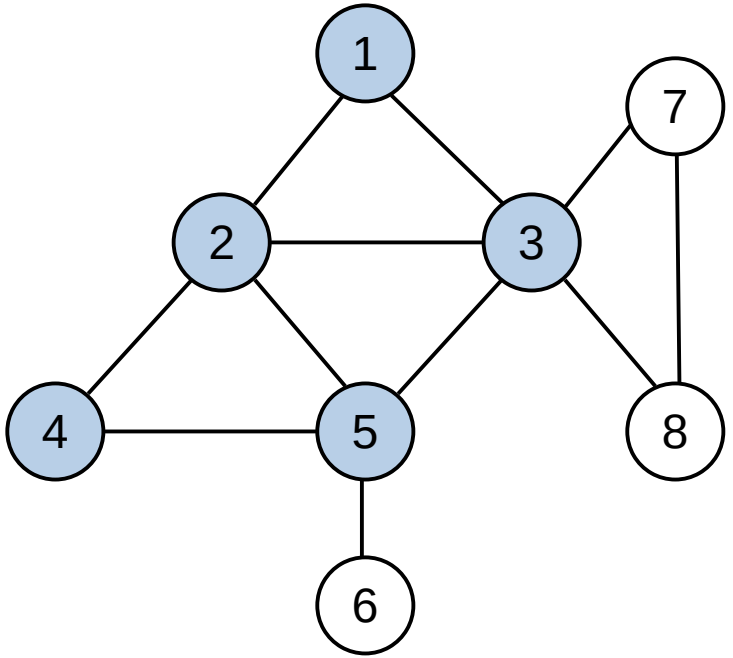
Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, 5, 4

Now, we've hit a dead end, let's backtrack



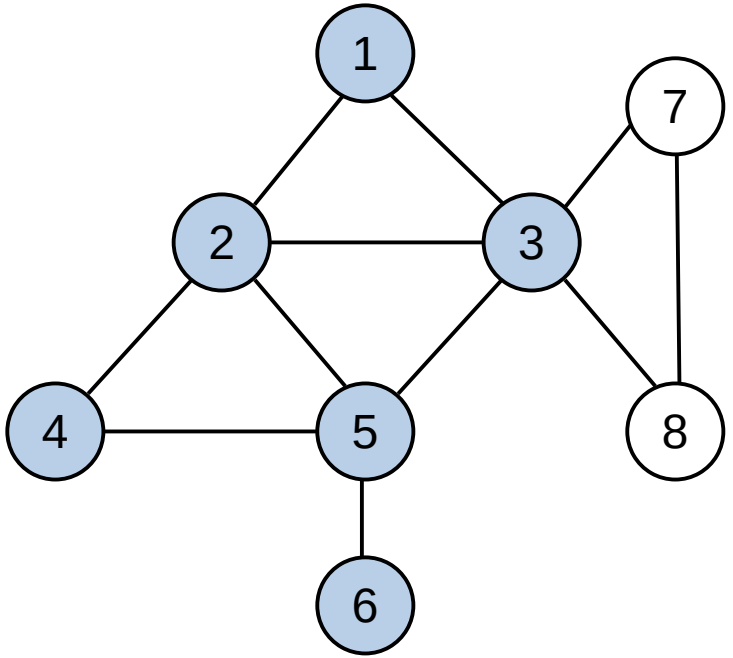
Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, 5, (6), (4)

Now, we've hit a dead end, let's backtrack



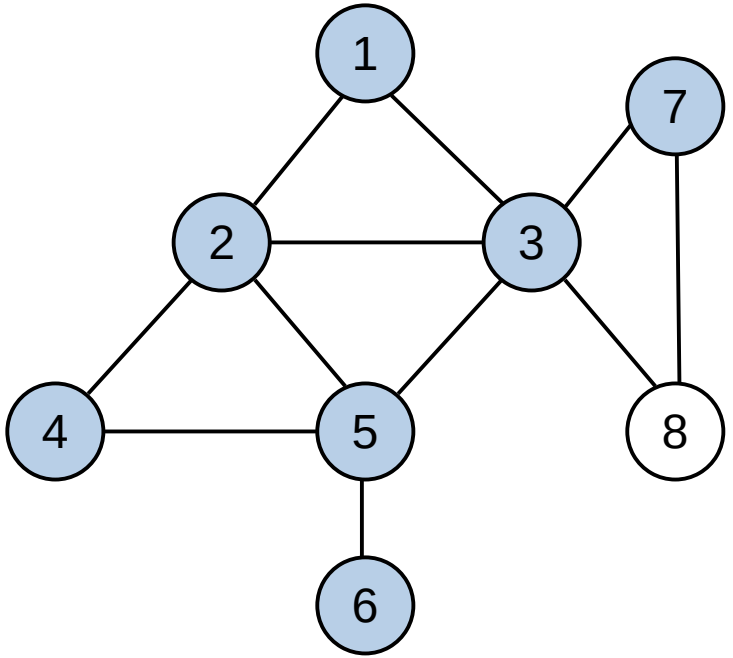
Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, (7), (5, (6), (4))

Now, we've hit a dead end, let's backtrack



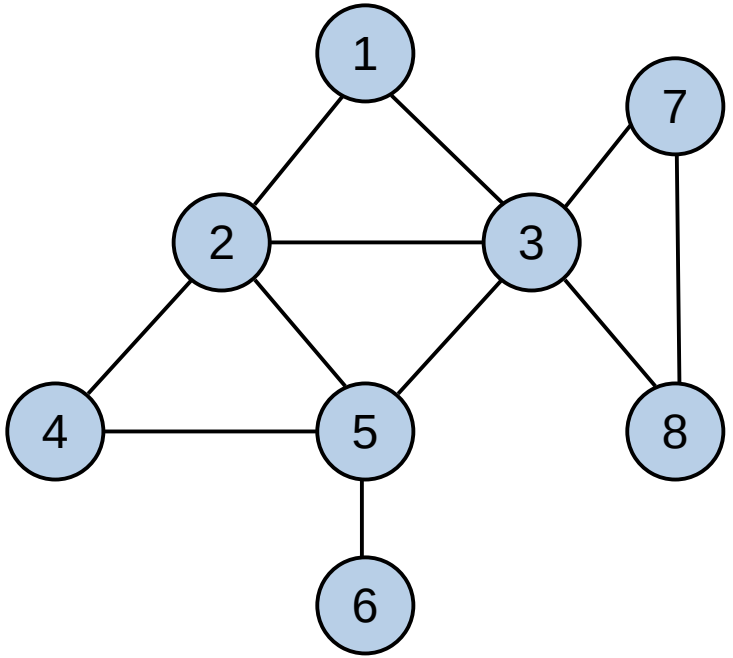
Depth-First Search

Keep exploring from the most recently added vertex until you hit a dead end, then backtrack

Let's start at vertex 1

Order visited: 1, 2, 3, (7, 8), (5, (6), (4))

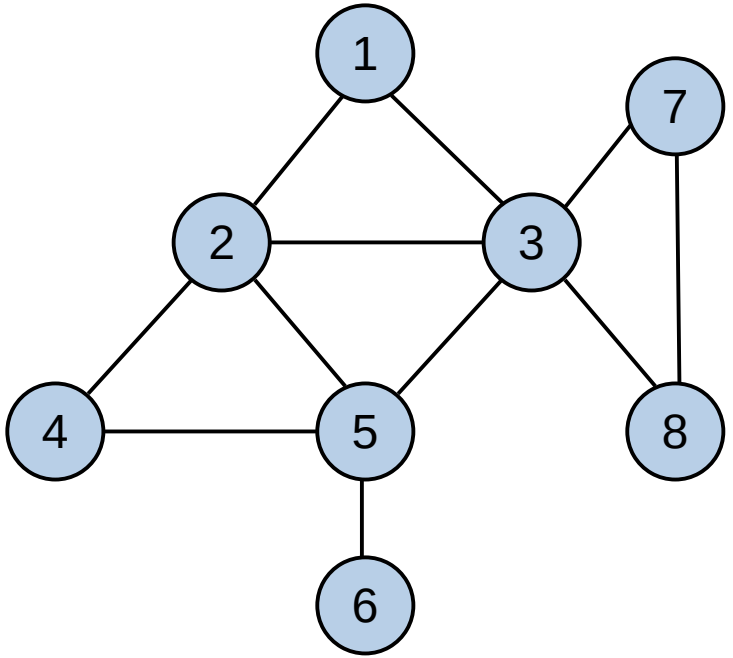
Now, we've hit a dead end, let's backtrack



Depth-First Search

Complexity: $O(n+m)$

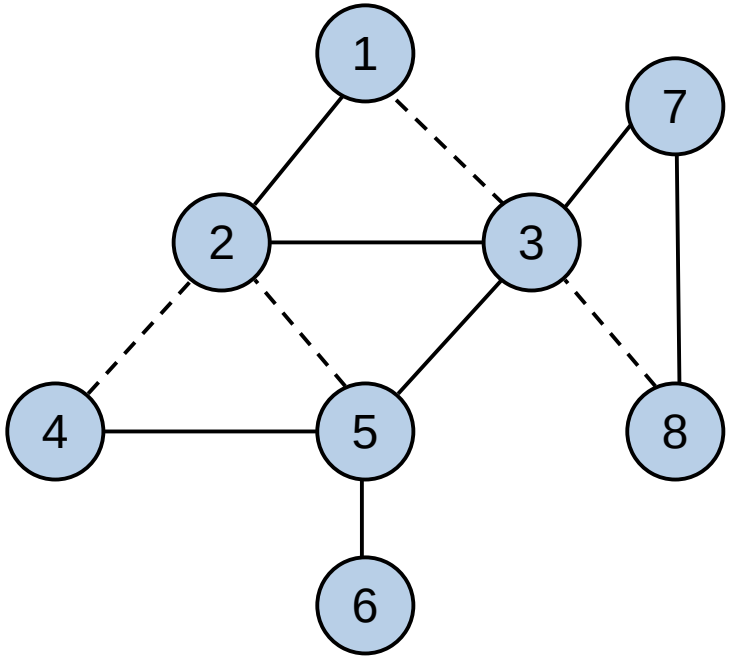
- # vertices + # edges
- Efficiently implement using a stack



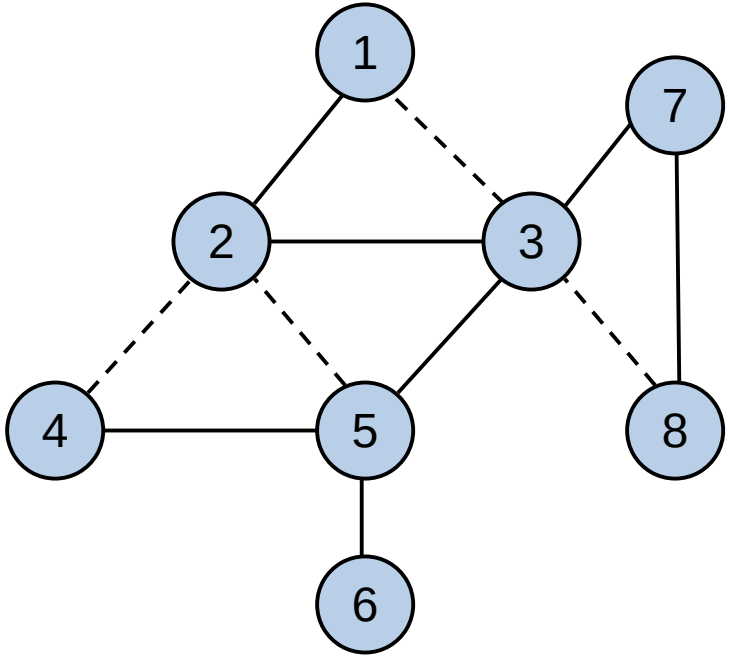
DFS Tree

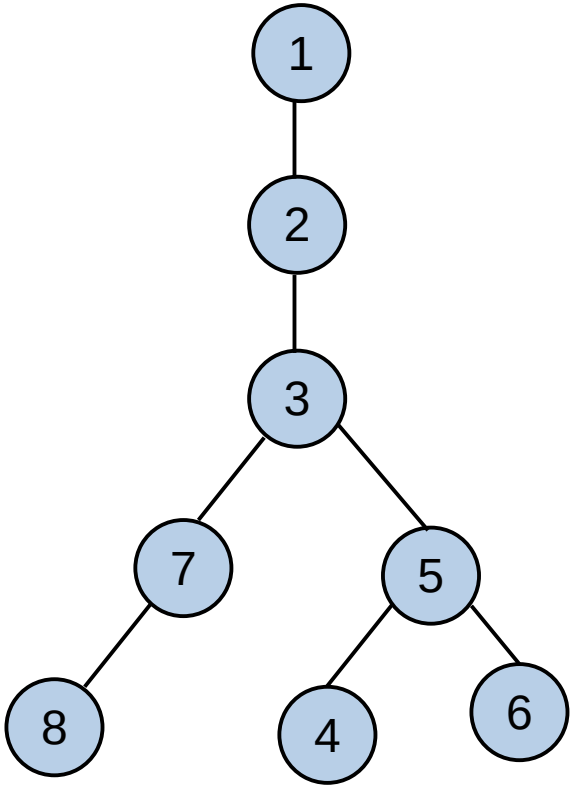
Order visited: 1, 2, 3, (7, 8), (5, (6), (4))

Keep edge uv if v was first explored from u

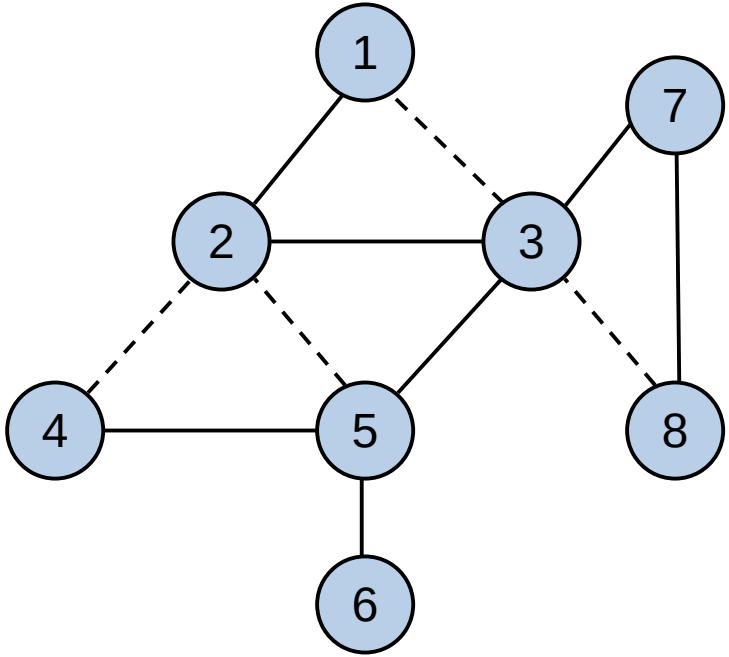


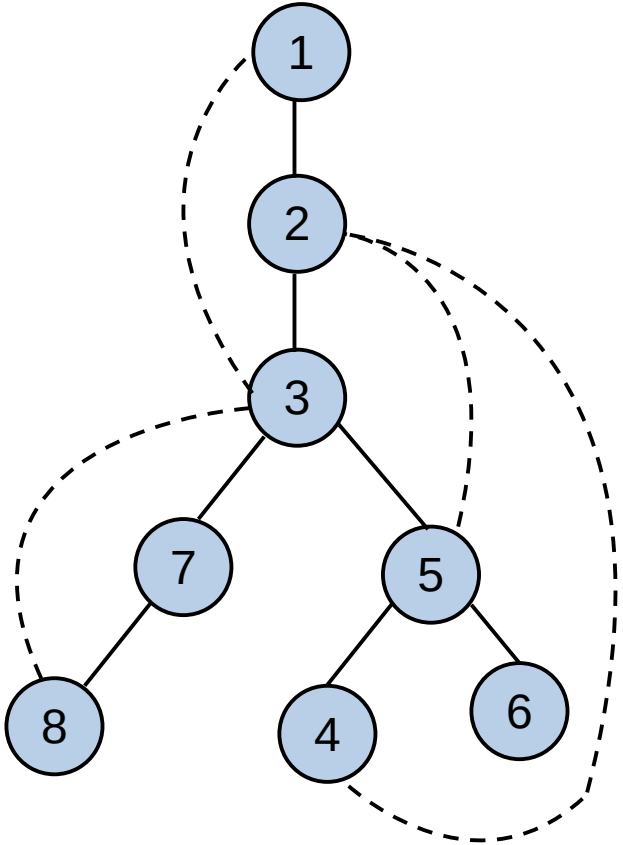
DFS Tree





DFS Tree



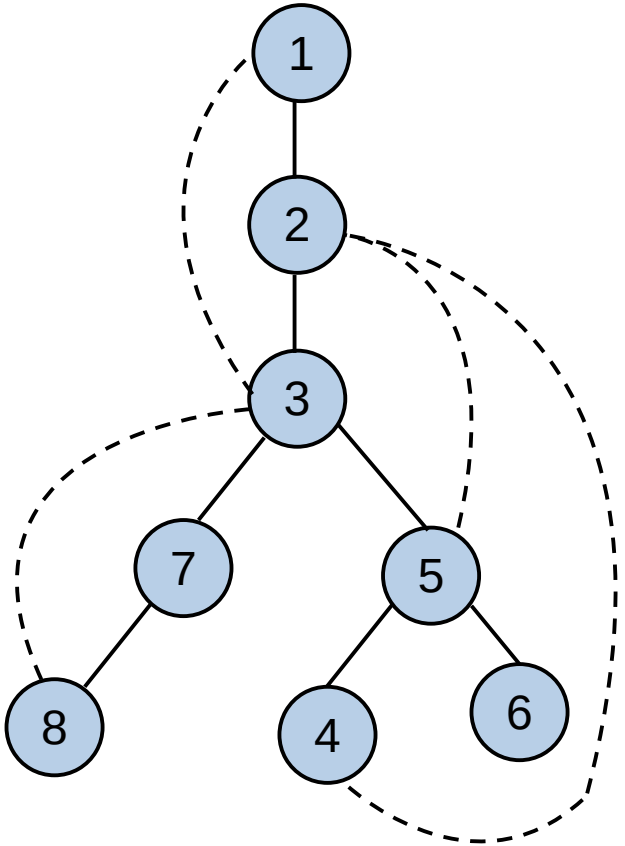


DFS Tree

Order visited: 1, 2, 3, (7, 8), (5, (6), (4))

Keep edge uv if v was first explored from u

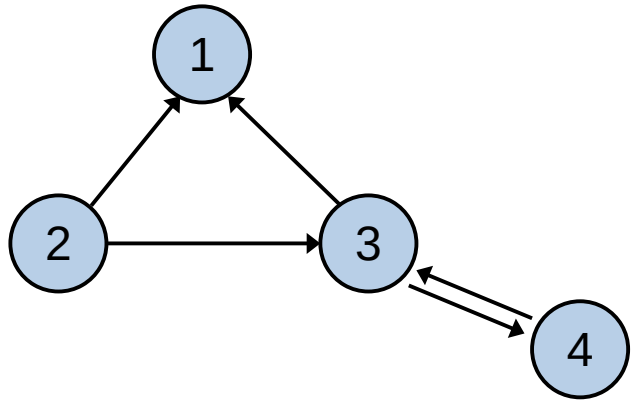
Non-tree edges in G lead to ancestors



Directed Graphs

`G = (V,E)`

- `(u,v) ∈ E` is a directed edge
- We say that u points to v
- `e = (u,v)` leaves u, enters v, is an outgoing edge from u, is an incoming edge to v



Directed Graphs

Most definitions extend naturally from undirected to directed graphs by mapping the word "edge" to "directed edge"

- A **directed path** is a sequence of vertices $P = v_1, v_2, \ldots, v_k$ such that $v_i v_{i+1}$ is an edge from v_i to v_{i+1}

Directed Graphs

Most definitions extend naturally from undirected to directed graphs by mapping the word "edge" to "directed edge"

- A **directed path** is a sequence of vertices $P = v_1, v_2, \ldots, v_k$ such that $v_i v_{i+1}$ is an edge from v_i to v_{i+1}
- When referring to a directed graph, when we say "path" or "cycle", we mean directed path and directed cycle

Directed Graphs

Most definitions extend naturally from undirected to directed graphs by mapping the word "edge" to "directed edge"

- A **directed path** is a sequence of vertices $P = v_1, v_2, \ldots, v_k$ such that $v_i v_{i+1}$ is an edge from v_i to v_{i+1}
- When referring to a directed graph, when we say "path" or "cycle", we mean directed path and directed cycle
- What about **connectedness**? This is more complex, since there may be a path from u to v but not vice versa

Modeling Networks

For each of the following, determine if it a directed or undirected graph fits better:

- Facebook and LinkedIn
- X and Instagram
- One-way roads between buildings with a city
- Interstate highways connecting cities

Modeling Networks

For each of the following, determine if it a directed or undirected graph fits better:

- Facebook and LinkedIn **Undirected**
- X and Instagram **Directed**
- One-way roads between buildings with a city **Directed**
- Interstate highways connecting cities **Undirected**

Digraph Traversal

Directed Reachability Problem

- Starting from vertex s, can we reach vertex v?
- Finding all vertices reachable from s
- What is the shortest path from s to v?

Digraph Traversal

Directed Reachability Problem

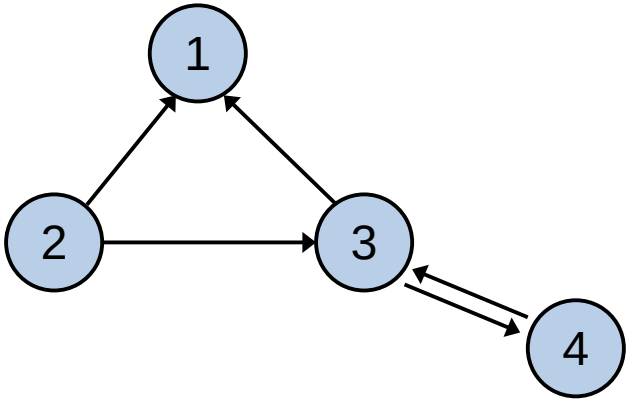
- Starting from vertex s, can we reach vertex v?
- Finding all vertices reachable from s
- What is the shortest path from s to v?

BFS/DFS naturally extend to directed graphs

- Replace "neighbors" with "outgoing neighbors"

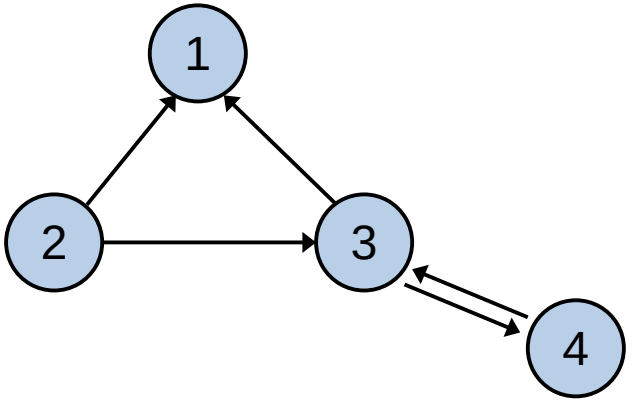
Digraph Traversal

- How can we find all vertices `v` such that there exists a path from `v` to vertex `t`?
- For example, how can we find all the vertices that can reach 1?



Digraph Traversal

- How can we find all vertices `v` such that there exists a path from `v` to vertex `t`?
- For example, how can we find all the vertices that can reach 1?

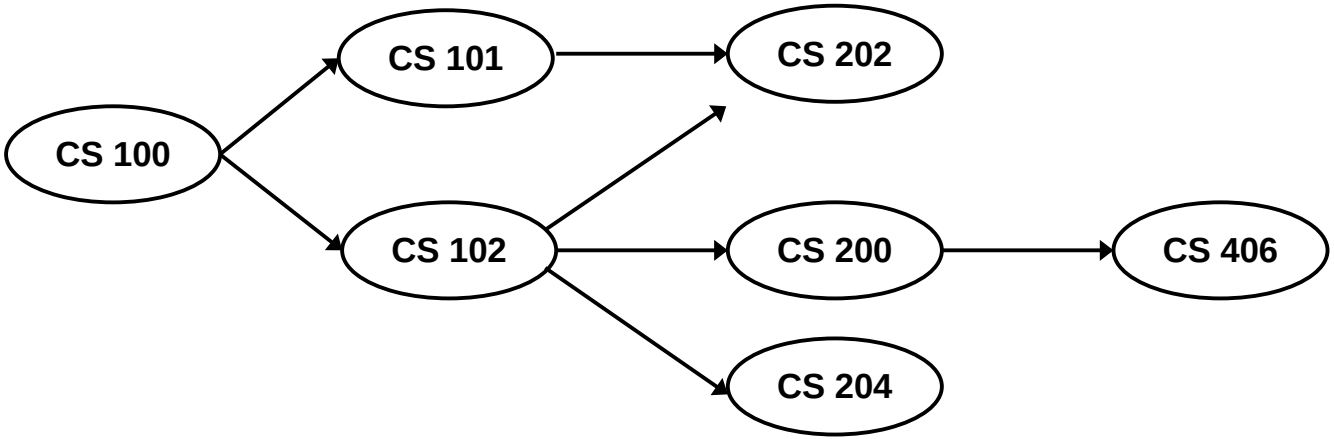


- Run BFS following edges in reverse direction
  - Change "outgoing" to "incoming"

Directed Acyclic Graphs

A **DAG** is a directed graph with with no cycles

- Modeling dependencies, e.g., course prerequisites and software requirements



Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Is a topological order possible for any directed graph?

Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

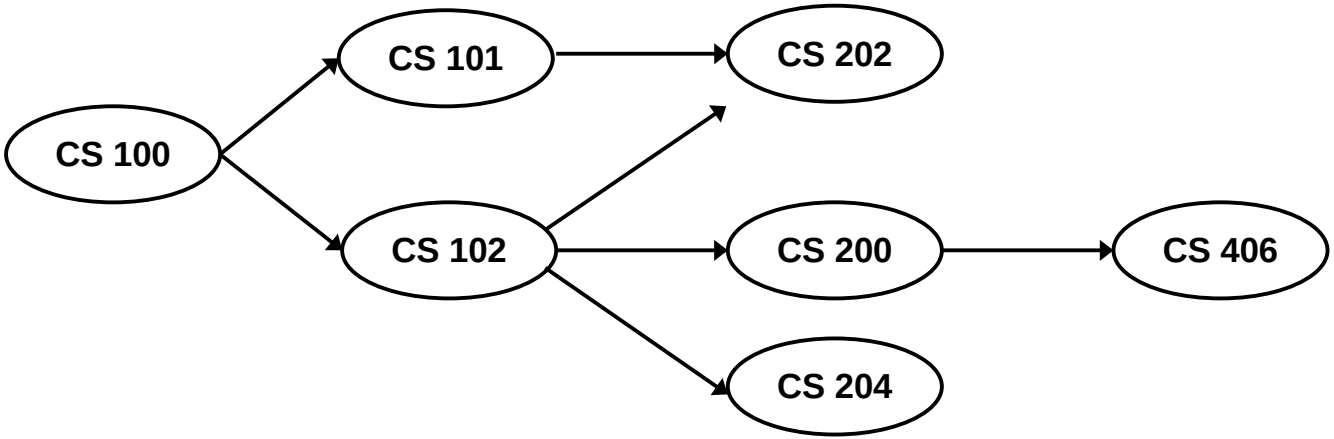
- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Is a topological order possible for any directed graph?

- No, a digraph has a topo ordering if and only if it is a DAG

Exercise

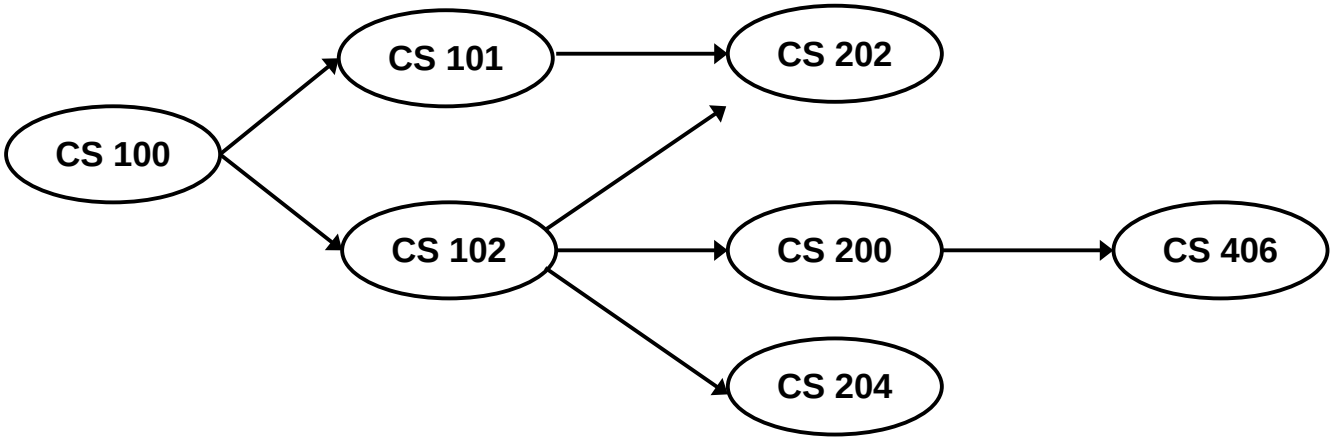
Find a topological order for the following digraph



Exercise

Find a topological order for the following digraph

- 100, 102, 204, 200, 406, 101, 202



Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

while there are vertices remaining:

- find vertex v with no incoming edges
- place v next in the order
- delete v from `D`

Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

while there are vertices remaining:

- find vertex v with no incoming edges
- place v next in the order
- delete v from `D`

Running time: `O(n+m)`

Next Time

- Get up to speed on recursion